Cloud File Storage System

A Dropbox-like platform built on Kubernetes, NGINX, and MinIO

Practical Group Assignment

Group size	2–3 students
Submission	Git repository + PDF report + demonstration

ABSTRACT

In this assignment, student groups will design, implement, and deploy a cloud-native file storage platform inspired by services such as Dropbox or Google Drive. The system is built from scratch using industry-standard technologies: Kubernetes for container orchestration, NGINX as a reverse proxy with SSL termination and authentication, MinIO (or RustFS) as an S3-compatible object storage backend, Prometheus and Grafana for observability, and Terraform for infrastructure-as-code provisioning. No graphical interface is required. The objective is to consolidate practical skills in cloud infrastructure, security, scalability, and operational monitoring.

1 CONTEXT AND OBJECTIVE

This project asks student groups to build and deploy a file-storage API that mirrors the core functionality of commercial cloud storage services. Users who have been authenticated may **upload**, **download**, and **share** files through a RESTful HTTP API. The system must run inside a Kubernetes cluster, persist data in an S3-compatible object store, and expose a single secured entry-point via NGINX.

The assignment is deliberately scoped to the *infrastructure and backend* layers. Designing a graphical user interface is explicitly out of scope; tools such as `curl`, `httpie`, or Postman are sufficient for all demonstrations and evaluations.

Central Goal

Build and deploy a file-storage REST API running on Kubernetes, backed by MinIO/RustFS for object storage, exposed through NGINX with HTTPS and authentication. All infrastructure must be provisioned and torn down exclusively via `terraform apply` / `terraform destroy`.

2 SYSTEM ARCHITECTURE

Figure 1 presents the high-level architecture. Each numbered layer is described in detail in Section 3. Data flows from left to right: the client reaches the NGINX gateway, which enforces authentication and rate-limiting before forwarding requests to the Kubernetes-hosted API pods. The API interacts with MinIO for object persistence. A separate observability stack collects metrics, logs, and alerts.

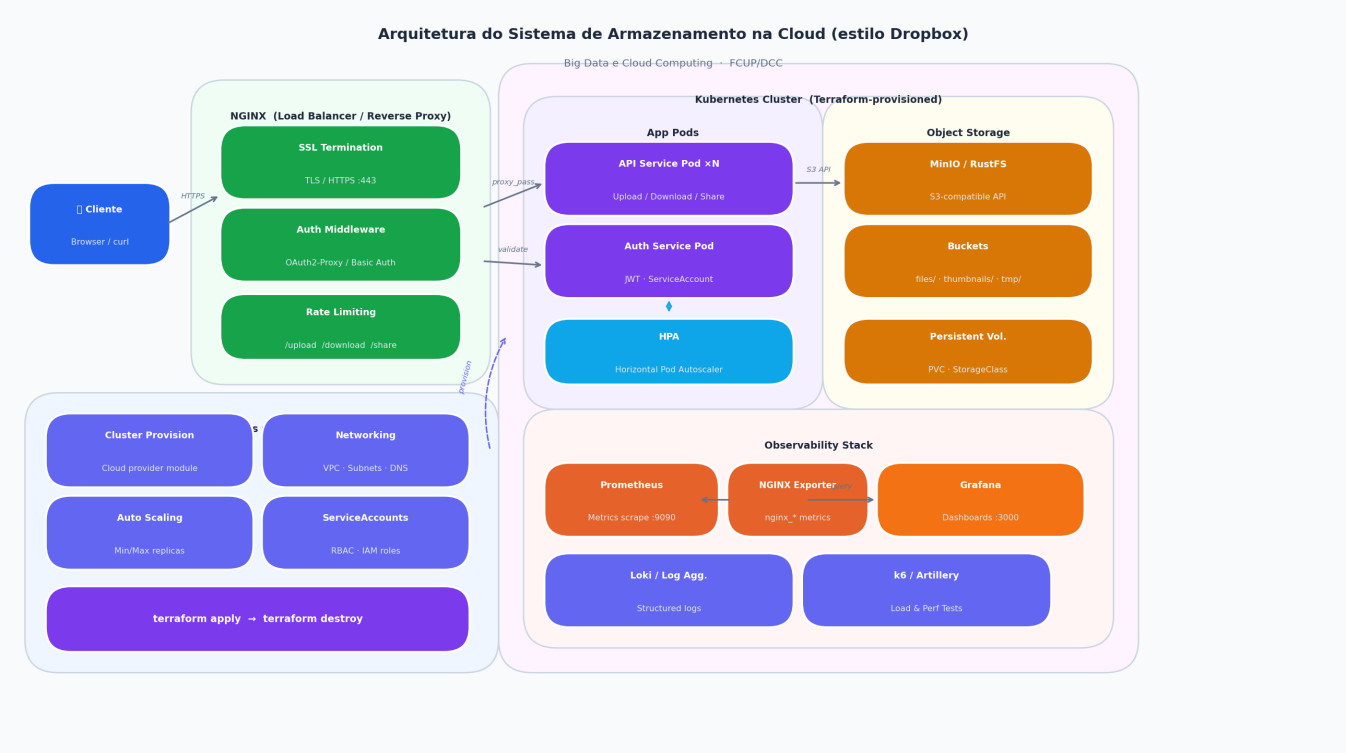


Figure 1 — High-level system architecture. Arrows denote data flow and provisioning dependencies.

2.1 Component Overview

COMPONENT	ROLE	TECHNOLOGY
NGINX	Single entry point: SSL termination, authentication gateway, rate limiting, reverse proxy	`nginx:latest / openresty`
OAuth2-Proxy	Authentication middleware; validates tokens before forwarding requests. Alternative: HTTP Basic Auth	`oauth2-proxy / htpasswd`
API Service	Business logic: upload, download, share, per-user quota enforcement	`Python/FastAPI or Go`
MinIO / RustFS	S3-compatible object storage; persists all user files across pod restarts	`minio:latest / rustfs`
Prometheus	Metrics collection via scrape jobs targeting all system components	`prom/prometheus`
NGINX Exporter	Exports NGINX runtime metrics to Prometheus at :9113/metrics	`nginx/nginx-prometheus-exporter`
Grafana	Dashboard visualisation and alert management	`grafana/grafana`
HPA	Horizontal Pod Autoscaler: scales API pods between 2 and 10 replicas	`K8s HorizontalPodAutoscaler`
Terraform	Provisions the entire infrastructure (cluster, networking, IAM, storage)	`hashicorp/terraform`
k6 / Artillery	Load and performance testing of the API under concurrent users	`grafana/k6 or artillery`

2.2 HTTP Request Flow

Figure 2 details the path of an individual HTTP request through the NGINX gateway. Three sequential decisions govern whether the request is forwarded to the API or rejected: TLS validation, authentication, and rate-limit enforcement.

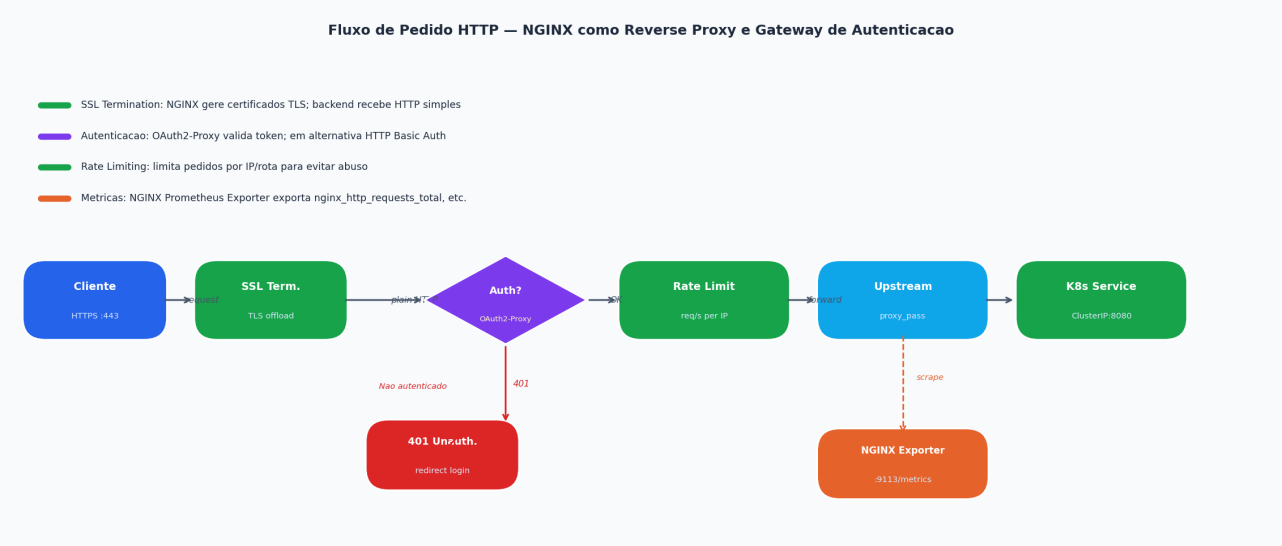


Figure 2 — Detailed HTTP request flow through the NGINX gateway. An unauthenticated request is rejected with HTTP 401 before reaching the upstream service.

3 DETAILED COMPONENT DESCRIPTION

3.1 NGINX — Load Balancer, SSL, and Authentication Gateway

NGINX acts as the system's sole external entry point. Its responsibilities span four distinct layers:

- **SSL/TLS Termination.** NGINX manages TLS certificates (self-signed in development, cert-manager in production) and accepts HTTPS on port 443, passing plain HTTP internally.
- **Authentication.** Requests are validated by an OAuth2-Proxy sidecar before forwarding. A simpler alternative is HTTP Basic Auth configured with `htpasswd`.
- **Rate Limiting.** Per-IP and per-route limits are enforced with `limit_req_zone` / `limit_req` directives, protecting against abuse and brute-force attacks.
- **Metrics Export.** The NGINX Prometheus Exporter exposes runtime metrics at `:9113/metrics` (connections, request rate, upstream latency) for consumption by Prometheus.

3.2 MinIO / RustFS — Object Storage

MinIO provides an S3-compatible API, allowing the application layer to interact with storage using the standard AWS SDK without modification.

- **Bucket layout.** Three buckets are required: `files/` (user data), `thumbnails/` (optional previews), `tmp/` (incomplete uploads).
- **Persistence.** MinIO must be backed by a PersistentVolumeClaim so that data survives pod restarts.
- **Presigned URLs.** File sharing is implemented by generating presigned URLs with a configurable TTL, exposing no credentials to the client.
- **Service account.** A dedicated MinIO service account with a minimal access policy must be used; the root account must not be used by the API.

3.3 Kubernetes — Orchestration and Autoscaling

The Kubernetes cluster provides the execution environment for all microservices. The following primitives must be employed:

- **Deployments.** The API runs with at least 2 replicas for fault tolerance.

- **Services.** Internal ClusterIP services route traffic between NGINX and the API, and between the API and MinIO.
- **ConfigMaps and Secrets.** Non-sensitive configuration in ConfigMaps; all credentials and API keys in Kubernetes Secrets (never committed to the repository in plaintext).
- **ServiceAccounts.** Each workload runs under its own Kubernetes ServiceAccount with the minimum required RBAC permissions (principle of least privilege).
- **Horizontal Pod Autoscaler.** The HPA scales the API between 2 and 10 replicas with a CPU target of 60 %, or an equivalent custom metric via the Prometheus Adapter.
- **Resource requests and limits.** All containers must declare `resources.requests` and `resources.limits` so that the scheduler can plan and enforce resource boundaries.

3.4 Terraform — Infrastructure as Code

All infrastructure is provisioned exclusively through Terraform, ensuring reproducibility, auditability, and team collaboration via version-controlled configuration.

- **Module structure.** Code is organised into reusable modules: `network/`, `cluster/`, `storage/`, `iam/`.
- **Variables and outputs.** All tunable parameters (cluster size, region, replica counts) are declared in `variables.tf`; endpoints and resource IDs are exported via `outputs.tf`.
- **Remote state.** Terraform state must be stored remotely (e.g., S3 bucket or Terraform Cloud) to support team collaboration without state conflicts.
- **Autoscaling.** Terraform provisions the HPA manifest and the node group's cluster-autoscaler policy so that new nodes appear automatically when pod demand grows.
- **Full lifecycle.** The system must be fully provisioned with `terraform apply` and fully removed with `terraform destroy`, without any manual intervention.

3.5 Observability — Prometheus, Grafana, and Logs

Production systems require full observability across metrics, logs, and alerts.

- **Prometheus.** Scrape jobs must target: API pods (via `prometheus.io/scrape` annotations), NGINX Exporter (:9113), MinIO health endpoint (:9000), and *kube-state-metrics*.
- **NGINX Prometheus Exporter.** Exposes key counters and gauges: `nginx_http_requests_total`, `nginx_connections_active`, and upstream response time histograms.

- **Grafana.** A dashboard with at least six panels is required: request rate (RPS), latency P50/P95/P99, HTTP error rate (4xx/5xx), pod CPU and memory utilisation, and MinIO I/O throughput.
- **Log aggregation.** Structured JSON logs from all components must be collected by Loki + Promtail (or an equivalent stack) and made searchable from within Grafana.
- **Alerting.** At least three Prometheus alerting rules are required: error rate > 5 %, P95 latency > 2 s, and pod CPU utilisation > 80 %.

4 REST API SPECIFICATION

The API is the system's central component. It must follow REST conventions and be documented with an OpenAPI/Swagger specification committed to the repository. No graphical interface is required; `curl` or Postman are sufficient for all demonstrations.

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/auth/login`	*Public*	Authenticate user; returns JWT or session cookie
POST	`/files/upload`	*Bearer JWT*	Upload file (multipart/form-data). Returns file_id and metadata
GET	`/files`	*Bearer JWT*	List all files owned by the authenticated user (paginated)
GET	`/files/{file_id}`	*Bearer JWT*	Download a file by its unique identifier
DELETE	`/files/{file_id}`	*Bearer JWT*	Permanently delete a file from object storage
POST	`/files/{file_id}/share`	*Bearer JWT*	Generate a presigned URL with configurable TTL for public sharing
GET	`/files/{file_id}/meta`	*Bearer JWT*	Return file metadata: name, size, MIME type, upload timestamp
GET	`/health`	*Public*	Kubernetes liveness probe endpoint; returns HTTP 200
GET	`/metrics`	*Network Policy*	Prometheus metrics scrape endpoint (internal cluster access only)

4.1 Expected Request/Response Format

All endpoints consume and produce `application/json` unless otherwise specified. File upload uses `multipart/form-data`. Errors follow RFC 7807 (Problem Details for HTTP APIs). Authentication tokens are passed in the `Authorization: Bearer <token>` header.

```
# Example: upload a file
curl -X POST https://<host>/files/upload \
  -H "Authorization: Bearer <jwt>" \
```

```
  -F "file=@report.pdf"

# Example: generate a presigned share link (TTL = 1 hour)
curl -X POST https://<host>/files/<id>/share \
  -H "Authorization: Bearer <jwt>" \
  -d '{"ttl_seconds": 3600}'
```

5 ASSIGNMENT TASKS

The assignment is divided into five tasks. Although each task builds upon the previous one, they are graded independently. Groups are strongly advised to follow the order below.

5.1 Task 1 — Base Infrastructure with Terraform

Provision the Kubernetes cluster and all required network resources using Terraform.

(1) **Network module.** VPC, public/private subnets, firewall rules, and DNS configuration.

(2) **Cluster module.** Kubernetes cluster (k3s or kind locally; EKS/GKE/AKS if cloud resources are available).

(3) **IAM and ServiceAccounts.** One Kubernetes ServiceAccount per workload with minimum necessary RBAC permissions.

(4) **Validation.** `terraform plan` completes without errors; `kubectl get nodes` shows all nodes in `Ready` state.

5.2 Task 2 — NGINX: Load Balancer, SSL, and Authentication

Configure NGINX as the system's sole external entry point.

(5) **SSL/TLS.** Terminate HTTPS at NGINX. In development, a self-signed certificate is acceptable; in production, use cert-manager. All HTTP requests must be permanently redirected to HTTPS.

(6) **Authentication.** Integrate OAuth2-Proxy with an OIDC provider (e.g., Keycloak, Authelia, or a public provider). Accepted alternative: HTTP Basic Auth via `htpasswd`.

(7) **Rate limiting.** Limit upload to 10 req/min per IP; download to 60 req/min. Configure zones with `limit_req_zone`.

(8) **NGINX Exporter.** Install the exporter and verify that `curl :9113/metrics` returns valid Prometheus exposition format.

(9) **Validation.** `curl -k https://<ip>/health` returns HTTP 200; a request without credentials returns HTTP 401.

5.3 Task 3 — File API and MinIO/RustFS

Implement the REST API and configure the object storage backend.

(10) **MinIO/RustFS.** Deploy using Helm or a plain YAML manifest. Create the required buckets, service account, and access policies.

(11) **API service.** Implement all endpoints from Section 4. The API must validate incoming JWT tokens, enforce per-user quotas, and interact with MinIO via the official SDK.

(12) **Kubernetes deployment.** Deploy with 2 replicas, liveness/readiness probes, ConfigMaps for non-sensitive configuration, and Kubernetes Secrets for all credentials.

(13) **HPA.** Configure a HorizontalPodAutoscaler with `minReplicas=2`, `maxReplicas=10`, and a CPU target of 60 %.

(14) **Validation.** Successfully upload, list, download, and delete a file via curl; verify that the presigned share URL is accessible without authentication within its TTL.

5.4 Task 4 — Observability

Implement the full observability stack.

(15) **Prometheus.** Configure scrape jobs for all components. Verify that API, NGINX, and MinIO metrics appear in the Prometheus UI.

(16) **Grafana.** Create the required dashboard (minimum six panels, as described in Section 3.5). Export the dashboard JSON and commit it to the repository.

(17) **Log aggregation.** Configure Loki + Promtail (or equivalent). Demonstrate log search by pod and by log level (ERROR, WARN) within Grafana.

(18) **Alerting.** Configure the three alerting rules listed in Section 3.5. Simulate an alert condition and verify that the alert fires and resolves correctly.

5.5 Task 5 — Performance Testing

Evaluate system behaviour under sustained load and verify autoscaling correctness.

(19) **Load script.** Write a k6 or Artillery script simulating 50 concurrent users performing upload and download operations for 5 minutes.

(20) **Metrics collection.** Record: mean and P95 latency, requests per second, error rate, and replica count over time.

(21) **Autoscaling verification.** Demonstrate that the HPA increases the replica count during the load test and scales back down afterwards.

(22) **Report inclusion.** Include Grafana screenshots and k6/Artillery output in the final report.

6 DELIVERABLES

All deliverables must be submitted via a Git repository (gitlab.up.pt) shared with the course instructor before the stated deadline. Every item below is mandatory.

6.1 Repository Structure *(suggestion)*

```
project-dropbox/
  terraform/          # All IaC code
    network/  cluster/  iam/
  k8s/                # Kubernetes manifests (Deployments, Services, HPA, Secrets
...)
  nginx/              # nginx.conf and local docker-compose for development
  api/                # API source code + Dockerfile
  observability/      # prometheus.yml, Grafana dashboards JSON, alerting rules
  tests/              # k6 / Artillery scripts
  docs/               # Final report (PDF)
  README.md           # Setup and usage instructions
```

6.2 Deliverable Checklist

- **Git repository.** Clean commit history following Conventional Commits conventions. No credentials or secret keys committed (use `.gitignore` and Kubernetes Secrets).
- **README.md.** Complete, step-by-step instructions to install and run the project from scratch, including all prerequisites, Terraform commands, and post-deploy validation steps.
- **Report (PDF, max. 20 pages).** Architecture design decisions, challenges encountered, an updated architecture diagram, and performance test results with analysis.
- **Demonstration.** A short video (max. 10 min) or live demo showing: file upload, download, sharing via presigned URL, Grafana dashboard, and HPA scaling in action.

7 FREQUENTLY ASKED QUESTIONS

7.1 Can I use a local Kubernetes cluster (kind, minikube, k3s)?

Yes. For local development, **kind** (Kubernetes IN Docker) or **k3s** are recommended. For the final demonstration, a real cloud cluster (EKS, GKE, or AKS) is preferred but not mandatory. Terraform configuration must be adapted to the chosen environment.

8.2 Do I need to build a graphical interface?

No. A graphical interface is explicitly out of scope. Evaluation is based entirely on the REST API, the cloud infrastructure, and the observability stack. Tools such as `curl`, `httpie`, or Postman are sufficient for all demonstrations.

8.3 Which programming language should I use for the API?

The choice of language is free. **Python with FastAPI** is recommended for its native integration with boto3 (the MinIO/S3 SDK) and `prometheus_client`. Go is also a valid choice with lower runtime overhead. The language selection has no bearing on grading.

8.4 MinIO or RustFS?

Both are accepted. MinIO is more mature and better documented. RustFS (`https://rustfs.com/`) is a newer fully open-source S3-compatible alternative. The choice does not affect grading.

8.5 How should I manage secrets and credentials?

Credentials must never appear in plaintext in the repository. Use Kubernetes Secrets, ideally encrypted at rest with Sealed Secrets or External Secrets Operator. In Terraform, mark sensitive variables with `sensitive = true` and store state remotely.

ACADEMIC INTEGRITY

All submitted work must be original and produced by the group. Consulting official documentation, tutorials, and publicly available examples is permitted, provided all sources are referenced. The use of generative AI tools to assist with code is permitted, but every group member must be able to explain any submitted code in full. Submitting work copied from other groups results in a grade of zero for all parties involved and may trigger formal academic misconduct proceedings.